Studio Nx

AI 구현 설명 86 — S1-12 오디오 회귀 테스트 기반 (PRD 00 §7.1)

2026-10-21

AI 구현 문서 86 — S1-12: 오디오 회귀 테스트 기반 (PRD 00 §7.1)

항목	내용
작업	S1-12 오디오 회귀 테스트 기반: 테스트 프로젝트 5개, 골든 WAV, 비교 스크립트
작성	Claude (AI), 2026-10-21, 사용자 검토
기반	S1-11c 병합(develop, PR #93), PRD 00 §7.1
프런트와 백엔드	앱 코드는 바꾸지 않았습니다. 모두 `studionx/tests/audio/` 아래의 새 파일입니다
함수 문서	JS 305 ~ 307 (3건)
시험	자체 시험 62건 통과 (기존 S1-11a, S1-11b, S1-11c 시험은 앱 코드를 안 바꿨으므로 다시 돌리지 않음)
이전 문서	85_s1-11c_sampler_midi.md, 84_s1-11b_wav_export.md

1. 실행 결과

DAW에서 가장 위험한 버그는 오류 없이 소리만 조용히 달라지는 것입니다. Sentry로는 잡히지 않습니다. 이제 명령 하나로 소리가 달라졌는지 확인할 수 있습니다.

```
> node studionx/tests/audio/run.mjs
프로젝트            결과  최대 차이   RMS dBFS  LUFS 차이  골든 LUFS  시간
effects_chain     PASS  0.000000    -inf      0.000     -6.7       0.2s
full_mix_tempo    PASS  0.000000    -inf      0.000     -15.8      0.2s
mixer_routing     PASS  0.000000    -162.9    0.000     -10.4      0.1s
sampler_drums     PASS  0.000000    -174.5    0.000     -16.5      0.1s
synth_basic       PASS  0.000000    -162.7    0.000     -16.2      0.2s
5/5 통과
```

확인한 장면	결과
아무것도 안 바꾸고 실행	5개 모두 통과합니다. 렌더링은 프로젝트마다 0.1초에서 0.3초가 걸립니다(Chrome을 띄우는 시간은 제외)
`synth.js`의 최고 음량 계수를 0.3에서 0.31(약 +3%)로 바꾸고 실행	Synth를 쓰는 두 프로젝트(`synth_basic`, `full_mix_tempo`)만 실패하고, 나머지 셋은 통과합니다. 실패한 렌더링은 `out/`에 WAV로 저장됩니다
원래대로 되돌리고 실행	다시 5개 모두 통과합니다
소리를 일부러 22가지로 바꿔서 비교	22가지 모두 실패합니다 (컷오프 5%, 마스터 -0.1dB, 노트 하나 반음, 템포 1BPM 등)
`--update "이유"`	골든을 새로 쓰고 이전과의 차이, 이유, 브라우저 버전을 변경 기록에 한 줄 남깁니다

2. 무엇을 만들었나

```
studionx/tests/audio/
  projects/*.json    테스트 프로젝트 5개 (마디와 박 단위, 시험용 소리는 수식으로 생성)
  golden/*.wav       기준 소리 5개 (32bit float 스테레오, 약 8MB)
  golden/CHANGELOG.md  골든이 바뀐 이유 기록
  lib/metrics.mjs    비교 지표: 최대 차이, 차이 RMS, LUFS                (305)
  lib/runner.mjs     정적 서버, 헤드리스 Chrome, 프로젝트 변환           (306)
```

```
lib/plugins.mjs     플러그인 정의를 DB 시드에서 읽기                    (306)
lib/wavio.mjs       골든 WAV 읽기와 쓰기                               (306)
run.mjs             비교와 골든 갱신 명령                              (307)
selftest.mjs        테스트 도구 자체 시험                              (307)
README.md           사용법
```

테스트 프로젝트	확인하는 것
`synth_basic`	Synth 두 층(saw 둘과 필터 LFO, 사인), 엔벨로프, 필터, LFO, 벨로시티, 레이어 합산
`sampler_drums`	Sampler One-Shot 드럼(킥, 스네어, 하이햇)과 Classic 베이스, 모드별 길이, 음높이, 겹침, 트랙 팬
`effects_chain`	이펙트 여섯 개(EQ, Compressor, Delay, Reverb, Gain, Limiter)를 차례로 통과, 루프 클립
`mixer_routing`	트랙 볼륨, 팬, 뮤트, 마스터 볼륨, 클립 gain, 오프셋, 반복, 음정 이동
`full_mix_tempo`	48kHz, 템포 변경(100에서 140 BPM), Synth와 Sampler와 이펙트를 합친 전체 믹스

3. 요구사항 대응

PRD 00 §7.1의 항목과 만든 것의 대응입니다.

PRD 항목	구현
방법: 테스트 프로젝트 세트를 오프라인 렌더링해 골든과 비교	앱의 `renderOffline`을 그대로 부릅니다 (306)
비교 기준: 샘플 최대 차이, RMS 차이, LUFS 차이	세 지표 모두 구현했고, 기본 허용은 0.001, -80 dBFS, 0.05 LU입니다 (305)
실행 시점: 로컬 명령 1개, Headless Chrome	`node studionx/tests/audio/run.mjs` (307)
기준 갱신: 의도한 경우만, 이유를 기록	`--update "이유"`가 이유를 필수로 받아 변경 기록에 남깁니다 (307)
보관 위치: `tests/audio/projects/`, `tests/audio/golden/`	저장소에 이미 있던 `studionx/tests/audio/` 아래 같은 이름의 폴더에 넣었습니다
WASM 비교에도 사용	이번에는 하지 않았습니다. 허용 오차를 프로젝트마다 덮어쓰는 `tolerance`를 열어 두었습니다
프로젝트 세트: 악기별, 이펙트별, 패처 병렬, 오토메이션, 템플릿	이번 범위(3~5개)는 악기별, 이펙트별, 믹서, 템포입니다. 패처와 오토메이션은 아직 앱에 없어서 뒤에 추가합니다

4. 왜 이렇게 (이번 작업의 결정)

결정	이유
앱 코드를 바꾸지 않고 새 파일로만 만들었습니다	회귀 테스트가 검사 대상을 건드리면 믿을 수 없습니다. 앱이 이미 `renderOffline`을 컨텍스트에 독립적으로 만들어 둔 덕분에(S1-11a, S1-11b) 그대로 부를 수 있었습니다.
서버와 DB 없이 돕니다	로그인, DB 상태, 네트워크가 테스트 결과에 섞이지 않습니다. 정적 서버만 띄우고, 플러그인 정의는 시드 SQL에서 읽습니다. 시드의 기본값이나 범위가 바뀌어 소리가 달라지는 것도 이 테스트가 잡습니다.
골든은 "맞는 소리"가 아니라 "현재 소리"입니다	이 테스트는 소리가 좋은지가 아니라 달라졌는지를 봅니다. S1-11c까지의 소리를 최초 기준으로 삼았고, 그 사실을 변경 기록 첫 줄에 적었습니다.
시험용 소리를 수식으로 만듭니다	샘플 바이너리를 저장소에 넣지 않아도 되고, JSON만 보고 소리의 의미를 알 수 있습니다.
골든을 32bit float로 저장합니다	손실이 없어서 비교 지표에 양자화 잡음이 섞이지 않습니다. 대신 파일이 크므로 프로젝트를 2마디로 짧게 했습니다(5개에 약 8MB).

결정	이유
허용 오차를 좁게 잡았습니다	같은 코드의 렌더링 차이는 1e-7(-140 dBFS) 수준입니다. 허용(0.001, -80 dBFS)은 그보다 1만 배 크므로 오탐이 없고, 3% 음량 차이나 0.1dB 차이도 잡습니다.
소리가 없으면 별도로 실패시킵니다	렌더링이 통째로 실패해 무음이 나오고 골든도 무음이면 차이가 0이라 통과합니다. 가장 위험한 사고라서 최소 피크를 따로 검사합니다.
자체 시험을 둡니다	통과하는 테스트로는 허용 오차가 너무 넓은지 알 수 없습니다. 소리를 일부러 바꿔 보고 실패하는지 확인하는 민감도 시험으로 증명합니다.

5. 시험 중에 발견해서 고친 문제

문제	원인	고침
첫 실행에서 렌더링 뒤 프로그램이 `EPERM`으로 끝남	Windows에서 Chrome을 끈 직후 임시 폴더가 잠겨 있어 삭제가 거부됨	삭제 실패를 무시합니다. 다음 실행이 지우고 시작합니다
프로젝트 둘의 피크가 풀스케일(1.0)을 넘음	부동소수점 렌더링은 값이 유지되지만, 실제로 내보내면 잘리는 소리를 기준으로 삼으면 안 됨	마스터 볼륨을 내렸습니다. 이제 모든 프로젝트의 피크가 1.0 아래입니다
같은 세션에서 다시 렌더링하면 비트까지는 같지 않음	32bit float의 반올림으로 1e-7 안팎의 차이가 생김 (같은 Chrome이어도)	재현성 기대를 "완전히 같음"에서 "1e-6 이하"로 바꿨습니다. 허용 오차의 천분의 1입니다
이펙트 순서를 바꿔도 소리가 같음 (Delay와 Reverb)	둘 다 선형이고 시간에 따라 변하지 않는 필터라서 순서를 바꿔도 결과가 같은 것이 수학적으로 맞음	시험이 틀렸습니다. 비선형인 Compressor와 Delay를 바꾸도록 고쳤습니다
Limiter 천장을 바꿔도 소리가 같음	소리가 천장보다 작아서 Limiter가 아무것도 하지 않았음	프로젝트의 Gain을 +8dB로 키워 Limiter가 실제로 일하게 했습니다
무음을 붙인 LUFS 비교가 기대와 다름	표준 알고리즘은 소리와 맞닿은 경계 블록을 값에 넣어서 약 0.3 LU 달라짐	구현은 맞았고 기대가 틀렸습니다. "붙인 구간의 길이가 달라도 값이 같다"로 고쳤습니다

앱 코드의 오류는 없었고, 모두 테스트 도구와 프로젝트의 문제였습니다.

6. 검증 결과

확인	결과
자체 시험 `selftest.mjs`	62건 통과 (지표 21, 골든 16, 재현성 1, 민감도 22, 소리 없음 1, 프로젝트 수 1)
비교 `run.mjs`	5개 프로젝트 통과, 프로젝트당 렌더링 0.1초에서 0.3초
실제 코드 변경 시험	Synth 최고 음량 +3%를 Synth를 쓰는 두 프로젝트만 정확히 잡음, 되돌린 뒤 통과
두 번의 별도 Chrome 실행	골든과 최대 차이 0.000000 (6자리 아래는 1e-7 수준의 반올림)

골든을 만든 환경은 Chrome 155.0.8059.85입니다. 자체 시험은 골든 파일이 있어야 하고, 프로젝트와 골든은 건드리지 않습니다.

7. 알려진 한계

한계	내용
브라우저 버전에 묶여 있습니다	Chrome의 큰 버전이 바뀌면 오디오 엔진의 계산(ConvolverNode의 FFT, 압축기 등)이 조금 달라질 수 있습니다. 그때는 실패 내용이 허용 오차 근처인지 확인하고, 필요하면 골든을 갱신하되 변경 기록에 브라우저 버전을 남깁니다

한계	내용
Chrome 하나만 씁니다	Firefox와 Safari의 오디오 계산은 비교하지 않습니다. 앱이 지원할 브라우저가 정해지면 별도 골든이 필요할 수 있습니다
골든은 "현재 소리"입니다	이미 있던 소리의 결함은 기준에 들어 있습니다. 이 테스트는 그 결함이 더 나빠지거나 바뀌는 것만 알려줍니다
패처 병렬, 오토메이션, 템플릿은 없습니다	앱에 아직 없는 기능이라서 뒤에 프로젝트를 추가합니다
자동 실행(CI)에는 연결하지 않았습니다	지금은 로컬 명령입니다. 저장소에 CI가 생기면 한 줄로 연결할 수 있습니다 (종료 코드를 지원합니다)
렌더링 시간이 짧아서 성능 회귀는 못 봅니다	소리만 비교합니다

8. 남은 일과 다음 작업이 쓰는 곳

항목	내용
다음 작업	S1-13 MVP 안정화 (버그 수정, 브라우저 테스트, Sentry 오류 정리). 앱 코드를 고칠 때마다 이 테스트를 돌립니다
이후 작업	새 악기, 이펙트(Drum Rack, FM, Filter 등)를 만들 때마다 프로젝트를 하나씩 추가하고 `--update`로 골든을 만듭니다
WASM 전환(C++ 병행 트랙)	같은 프로젝트를 Web Audio 버전과 WASM 버전으로 렌더링해 비교하는 데 씁니다. 허용 오차는 `tolerance`로 넓힙니다

9. 자주 받을 질문 대비 (Q/A)

Q. 오디오 회귀 테스트가 왜 필요한가요? A. 소리 코드의 버그는 오류를 내지 않고 소리만 조금 달라집니다. Sentry로는 잡히지 않고, 사용자가 "뭔가 이상하다"고 느낄 때까지 모릅니다. 이 테스트는 같은 곡이 늘 같은 소리가 나는지 숫자로 확인합니다.

Q. 골든 WAV는 무엇인가요? A. 기준으로 삼은 소리 파일입니다. 테스트 프로젝트를 렌더링해 저장해 두고, 이후 렌더링이 이 파일과 같은지 비교합니다. 의도적으로 소리를 바꿨을 때만 새로 씁니다.

Q. 어떻게 비교하나요? A. 세 가지 숫자를 씁니다. 샘플 하나의 최대 차이, 차이 신호의 RMS(dBFS), 통합 라우드니스(LUFS)의 차이입니다. 모두 허용 오차 안이어야 통과합니다 (305).

Q. 테스트가 실패하면 어떻게 하나요? A. `out/` 폴더에 저장된 현재 렌더링을 들어 보고 의도한 변화인지 판단합니다. 의도하지 않았다면 코드를 고치고, 의도했다면 `--update "이유"`로 골든을 갱신합니다 (307).

Q. 테스트 도구가 거짓으로 통과하지는 않나요? A. 자체 시험(62건)이 그것을 확인합니다. 소리를 22가지로 일부러 바꿔서 모두 실패하는지, 소리가 아예 안 나는 경우를 잡는지 봅니다. 실제 코드를 바꿔서도 확인했습니다 (307).

Q. 서버를 켜야 하나요? A. 아닙니다. 정적 서버를 테스트가 알아서 띄우고, 플러그인 정의는 DB 시드 파일에서 읽습니다. Chrome만 있으면 됩니다.